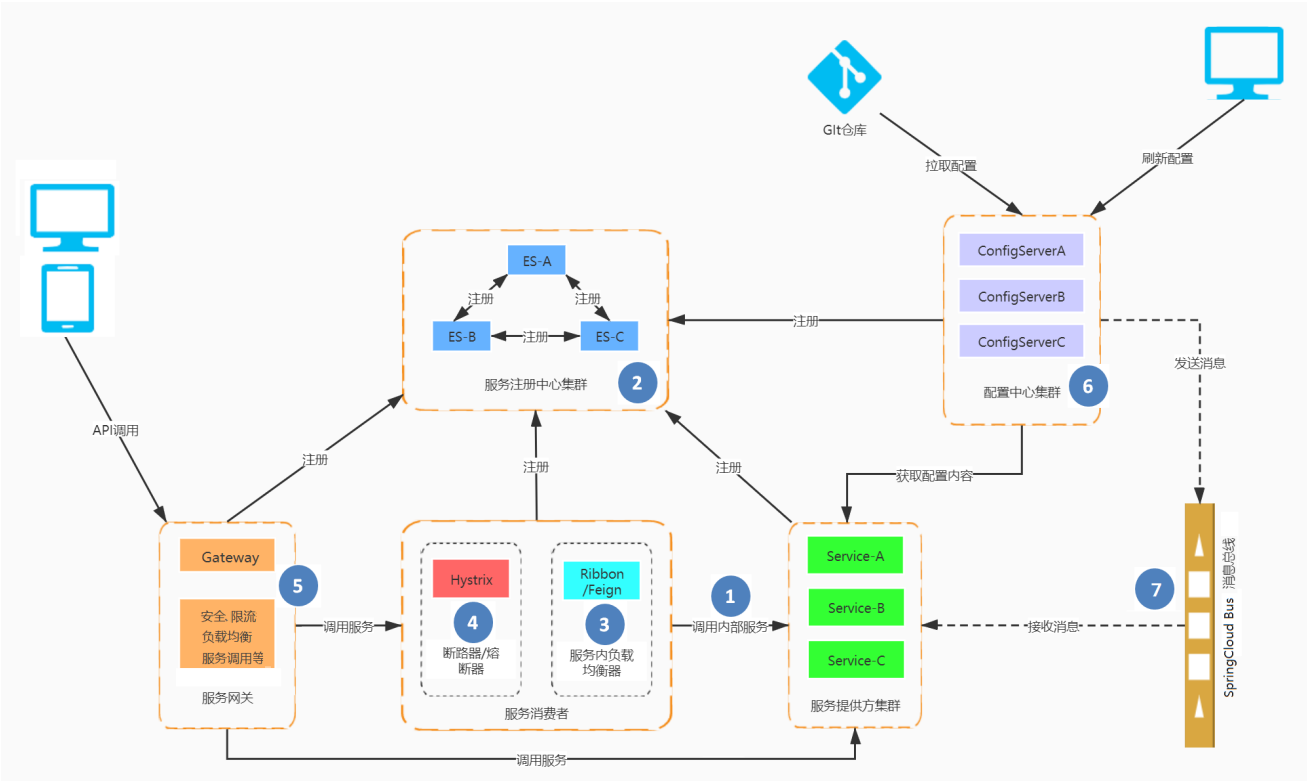
本章所讲内容为 Spring Cloud 架构图中的第六个部分。



1.配置中心简介

1.1.什么是配置中心

对于传统单体应用而言，常使用配置文件来管理所有配置。比如Springboot的application.yml文件。但是在微服务架构中全部手动修改的话会很麻烦而且不易维护。微服务的配置管理一般有以下需求：

1. 集中配置管理：一个微服务架构中会有成百上千个微服务，所以集中配置管理是很重要的。
2. 不同环境不同配置：有些配置在不同环境（开发、生产、测试）中是不同的。
3. 运行期间动态调整： 比如：运行中会根据各个微服务的负载情况动态调整数据源连接池大小等。
4. 配置修改后可自动更新：配置内容修改后，微服务可以自动更新配置。

综上所述，对于微服务架构来说，一套统一的、通用的管理配置中心是不可缺少的重要组成部分。

1.2.常用配置中心

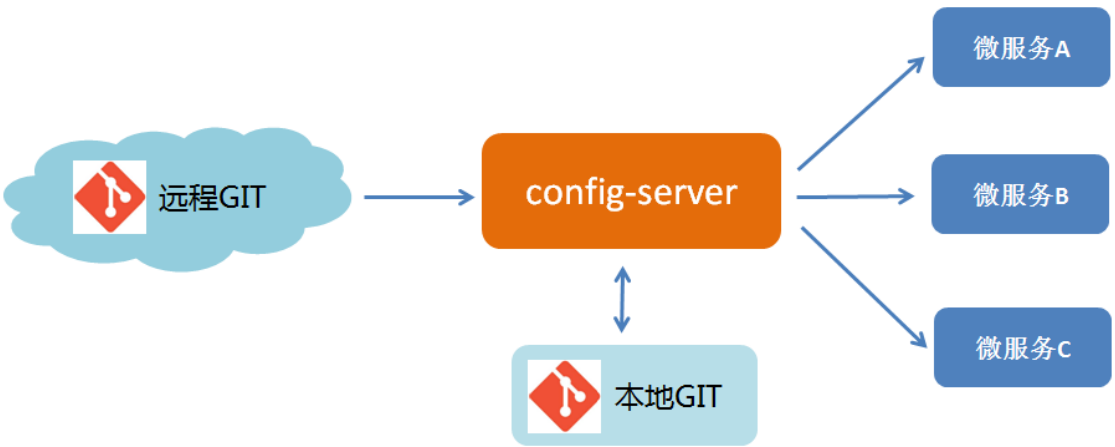
- Spring Cloud Config：2014年9月开源，Spring Cloud 生态组件，可以和Spring Cloud体系无缝整合。
- Apollo（阿波罗）：2016年5月开源，是携程框架部门研发的开源配置管理中心。能够集中管理应用在不同环境、不同集群的配置,配置修改后能够实时的推送到应用端,并且有关于权限管理、流程治理等功能。

- Nacos：2018年6月开源，阿里开源的配置中心。Nacos 提供了一组简单易用的特性集，帮助您实现动态服务发现、服务配置管理、服务及流量管理。

2. SpringCloud Config 配置中心

2.1.SCC简介

SpringCloud Config项目是一个解决分布式系统的配置管理方案。它包含了Client和Server两个部分，server默认使用git来提供配置文件的存储、以接口的形式将配置文件的内容提供出去，client通过接口获取数据、并依据此数据初始化自己的应用。



2.2.SCC实例

2.2.1.搭建远程GIT仓库

SCC Server 默认使用Git存储配置文件，当然也可以使用SVN。这里使用Git作为学习环境。

如果直接使用GitHub的话，国内用户经常会遇到无法连接或者网速过慢的问题。所以，这里我们使用国内的Git托管服务：码云。

1. 访问码云官网：https://gitee.com/ 按照码云官网的要求注册用户并登陆。
2. 新建仓库

3. 添加仓库信息：添加仓库名称，选择公开即可，然后点击最下方按钮“创建”

4. 初始化readme文件

> - 第一个红框内就是当前仓库地址
> - 此时可以点击第二个红框，初始化readme文件。

5. 仓库管理

> 可以在左上角的“管理”选项中，对本仓库进行清理、删除等操作。

2.2.2.上传配置文件

将需要进行集中管理的配置文件，上传到上面创建好的仓库中。

> 上传文件命名规则：
>
> {application}-{profile}.yml （或：{application}-{profile}.properties）
>
> application为应用名称、profile指的开发环境（用于区分开发环境（dev），测试环境（test）、生产环境（pro）等）

这里我们将服务提供者provider_server_11000的配置文件改名为：provider_server_11000-dev.yml；服务提供者provider_server_11001的配置文件改名为：provider_server_11001-dev.yml

然后上传到远程Git仓库中。

1. 进入上传文件界面

2. 拖拽文件上传，然后点击最下方按钮“提交”

3. 上传成功

注意：上传配置文件的字符编码集要正确（本项目中一律使用utf-8），否则不能正确识别。

2.2.3.创建SCC服务端

1. 在父工程下，创建 Maven Module 子工程（工程名：config_server_15000；Packaging：jar）
2. 在pom.xml文件中添加依赖

```
<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-config</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
    </dependency>

    <!--热部署 gav -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
</dependencies>
```

3. 创建主启动类

```
package com.neusoft;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;
```

```
@SpringBootApplication
@EnableConfigServer     //开启SCC服务器端注解
public class MyApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

4. 创建application.yml文件

```
server:
    port: 15000

spring:
    application:
        name: config-server
    cloud:
        config:
            server:
                git:
                    uri: https://gitee.com/yuhongjun01/scc-test.git  #git仓库地址
```

5. 测试：启动项目，浏览器地址栏中输入：http://localhost:15000/provider_server_11000-dev.yml

> provider_server_11000-dev.yml 就是Git仓库中上传的文件

2.2.4.修改SCC客户端

1. 修改provider_server_11000工程和provider_server_11001工程，在pom.xml文件中添加SCC的依赖

```
<!--加入spring-cloud-config的依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

2. 删除application.yml文件
3. 创建bootstrap.yml文件

```yaml
spring:
    cloud:
        config:
            #应用名称。对应git中文件名称的前半部分
            name: provider_server_11000
            #开发环境。对应git中文件名称的后半部分
            profile: dev
            #git分支
            label: master
            #config server的请求地址
            uri: http://localhost:15000
```

> - bootstrap.yml用来在程序引导时执行，应用于更加早期配置信息读取。也就是说：bootstrap.yml配置文件在application.yml配置文件之前被加载。
> - bootstrap.yml 用于系统级别的参数配置，这些参数一般是不会变动的。application.yml 用于定义应用级别的参数配置。
> - bootstrap.yml的典型应用场景就是：配置信息一般是从 config server中加载。为了取得配置信息，需要一些提早的引导配置。因此，把 config server 信息放在 bootstrap.yml，用来加载在这个时期真正需要的配置信息。

4. 重启测试：

   可以看到，即使没有了application.yml文件，微服务也可以正常工作。

   > 需要手动重启并且等待一段时间。因为config server需要从git上下载application.yml配置文件，并且放在微服务的缓存中。

2.3.高可用配置中心

如同我们前面讲过的高可用注册中心一样的原理，为了保证配置中心的高可用性，也应该实现配置中心集群。

Spring cloud提供了这样的解决方案。我们只需将config server端当成一个服务，注册到Eureka中，然后让微服务去Eureka中获取配置信息即可。

2.3.1.创建配置中心集群

1. 在父工程下，创建 Maven Module 子工程（工程名：config_server_15001；Packaging：jar）
2. 此工程除了端口需要修改之外，其余内容与上面创建的工程一样

2.3.2.集群注册到Eureka

1. 给配置中心集群中的子工程添加Eureka客户端依赖

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

2. 给配置中心集群中的子工程中的bootstrap.yml中添加Eureka配置信息

```
#eureka配置
eureka:
    client:
        service-url:
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
    instance:
        prefer-ip-address: true        #使用ip地址注册
        instance-id: ${spring.cloud.client.ip-address}:${server.port}
```

> 此时，可以在Eureka控制界面查看，配置中心集群中的子工程是否已经注册到Eureka上。

2.3.3.从Eureka中读取配置信息

配置中心集群已经注册给Eureka后，微服务就可以从Eureka中获取配置信息了。

修改provider_server_11000工程和provider_server_11001工程中的bootstrap.yml文件：

```
spring:
    cloud:
        config:
            #应用名称。对应git中文件名称的前半部分
            name: provider_server_11000
            #开发环境。对应git中文件名称的后半部分
            profile: dev
            #git分支
            label: master
            #config server的请求地址
            #uri: http://localhost:15000
            discovery:
                enabled: true                  #开启服务发现
                service-id: config-server      #config server服务名

#eureka配置
eureka:
    client:
        service-url:
            #将自己注册给Eureka Server集群
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
    instance:

        prefer-ip-address: true        #使用ip地址注册
```

```
        instance-id: ${spring.cloud.client.ip-address}:${server.port}
```

> - 尽管远程Git上的配置文件中已经有了Eureka配置信息，但是这里仍然要再写一遍。目的是要先将自己注册给Eureka，以便获取Eureka上的配置信息
> - config server的请求地址应该是通过 discovery.enabled:true 来开启服务发现，然后通过service-id:config-server中设置的config server服务名，来获取Git上的配置信息。

此时就可以重启测试了。